Mini Project 2: Reliable Data Transfer (RDT) Report

Authors: Eric Cheung, Harry Kim
MP-Group: 26
Date: November 24, 2025

Project Overview

This project implements a custom pipelined **Reliable Data Transfer (RDT)** over UDP sockets. Unlike TCP, UDP does not provide reliability, ordering, or congestion control by default. Our implementation builds these features on top of UDP to achieve:

- **Connection-oriented communication**: A handshake mechanism ensures sessions are established before data transfer.
- **Reliability**: Lost or corrupted packets are detected and retransmitted using acknowledgment and timeout mechanisms.
- **Pipelining**: Multiple packets are sent without waiting for individual acknowledgments, managed through a sliding window.
- **Flow control**: The receiver advertises its buffer capacity to prevent overflow.
- **Congestion control**: The sender adapts its sending rate based on observed loss and network conditions.

Protocol Specification

Our Reliable Transfer Protocol (RTP) is designed to provide reliable, connection-oriented communication over UDP sockets. The protocol incorporates mechanisms for connection setup, packet reliability, pipelining, flow control, and congestion control. Below we outline each component.

Connection Establishment

To ensure reliable, session-based communication over UDP, our protocol begins with a connection setup phase that establishes state between the client and server before any data transfer occurs.

- **Handshake Procedure**
  - **Client** → **Server:** Connection request (SYN)
  - **Server** → **Client:** Acknowledgment (SYN-ACK)
  - **Client** → **Server:** Final acknowledgment (ACK) confirming the session
- **Teardown**
  - **Client** → **Server**: Termination request (FIN)
  - **Server** → **Client**: Acknowledgment (ACK), followed by its own termination request (FIN)
  - **Client** → **Server**: Final acknowledgment (ACK) confirming closure

> **Note:**
>
> In our current implementation, we assume a stable network environment without unusual delays or anomalies during the handshake. If the handshake fails (e.g., due to lost or delayed packets), the

> connection attempt is considered unsuccessful.
>
> This approach is not fully robust, but it simplifies the design for the scope of this project. The handshake timeout is configurable by the user, allowing flexibility in different testing environments.
>
> Future improvements could include retry mechanisms or adaptive timeout strategies to enhance resilience under adverse network conditions.

Reliability

Our protocol ensures reliable delivery of packets over UDP by combining acknowledgments, retransmission, timeouts and checksums:

- **Acknowledgments (ACKs)**
  - ACK packets carry `seq = 0` and may contain data.
  - ACKs are used solely by the sender to verify whether a packet has been received.
  - The receiver employs **cumulative acknowledgments**, meaning each ACK confirms receipt of all packets up to the highest in-order sequence number.
- **Retransmission Strategy**
  - The sender retransmits only the **last unacknowledged packet**, based on the base sequence number of the sliding window.
  - This minimizes redundant retransmissions and aligns with cumulative ACK behavior.
- **Timeouts**
  - Retransmission is triggered by a timeout associated with the **window base packet**.
  - The timeout value is **configurable by the user**, allowing adaptation to different network conditions.
  - The sender tracks only the timeout of the base packet, simplifying timer management.
- **Checksum Strategy**
  - We use **CRC32** to detect corruption.
  - The checksum is computed over the **header (with checksum field set to 0) + payload data**.
  - At the receiver, the computed checksum is compared against the transmitted value to verify integrity.

> **Note:**
>
> This design prioritizes simplicity and correctness. By relying on cumulative ACKs and retransmitting only the base packet, the protocol avoids unnecessary duplication but may be less efficient under high-loss conditions. The configurable timeout provides flexibility, and future improvements could include per-packet timers or selective retransmission for enhanced robustness.

Pipelining

Our protocol supports pipelined transmission, allowing multiple packets to be in flight simultaneously within the congestion window (`cwnd`) limit.

- **Congestion Window (cwnd)**
    - The sender may transmit multiple packets as long as the total outstanding data is within the `cwnd`.
    - The window slides forward when the **base sequence number** (lowest unacknowledged packet) is acknowledged.
    - The initial `cwnd` size starts at one **Maximum Segment Size (MSS)**, which is configurable by the user.
    - The `cwnd` then increases according to the congestion control algorithm (see **Congestion Control** section).
- **Receiver Window (rwnd)**
    - The receiver maintains a buffer defined by the **receiver window (`rwnd`)**, which specifies how much data can be buffered before delivery to the application.
    - The `rwnd` size is configurable, allowing flexibility in simulating different receiver capacities.
    - Buffered data is held until consumed by the application.
- **Application Consumption**
    - In this project, we simulate the application as **always consuming data immediately by default**, ensuring the buffer does not overflow.
    - A flag is available to optionally control the **consumption rate**, enabling scenarios where the application consumes data more slowly.

> **Note:**
>
> Future improvements could include adaptive buffer management.

Flow Control

Flow control ensures that the sender does not overwhelm the receiver's buffer capacity. Our protocol uses a receiver-advertised window (`rwnd`) to manage this process.

- **Receiver Window (RWND) Management**
    - The receiver sets `rwnd = 0` when its buffer size plus the size of the last received packet exceeds the configured `rwnd` capacity.
    - When this occurs, the receiver sends a **RWND packet** with `seq = 0` and no payload to notify the sender to stop transmitting.
    - Any further packets received from the sender while `rwnd = 0` are silently dropped.
- **Probing Mechanism**
    - The sender periodically transmits a **PROBE message** with `seq = 0` and no payload to check if the receiver buffer has become available again.
    - Upon receiving a probe, the receiver responds with an acknowledgment reflecting its current buffer state, or drop it.
    - It is valid for the sender to send a `seq = 0` packet, since this sequence number already triggers an ACK containing the receiver's oldest in-order sequence response.
- **Configurable Probe Rate**

- The sender probes the receiver at a **user-configurable rate**, allowing flexibility in different testing scenarios.
- This ensures the sender can resume transmission promptly once the receiver buffer is available.

> **Note:**
>
> This design provides a simple but effective mechanism to prevent buffer overflow at the receiver. By advertising `rwnd = 0` and dropping excess packets, the receiver enforces backpressure. The probing mechanism ensures that communication can resume once buffer space is freed. While effective for controlled testing, future improvements could include adaptive probing intervals or explicit buffer occupancy reporting for greater efficiency.

Congestion Control

Our protocol implements congestion control mechanisms inspired by TCP to balance throughput and fairness under varying network conditions.

- **Slow Start**
  - Transmission begins conservatively with a congestion window (`cwnd`) of one MSS (Maximum Segment Size).
  - During slow start, `cwnd` increases by **one MSS for each acknowledgment received**:

    ```
    cwnd += MSS
    ```

  - The **slow start threshold (`ssthresh`)** is configurable by the user.
- **Congestion Avoidance**
  - Once `cwnd` reaches the `ssthresh` value, the sender transitions into congestion avoidance.
  - In this phase, `cwnd` increases more gradually, approximating linear growth:

    ```
    cwnd += (MSS * MSS // cwnd)
    ```

- **Timeout Handling**
  - When a timeout occurs, the sender resets `cwnd` back to **1 MSS** (effectively restarting slow start).
  - The `ssthresh` is updated to `max(cwnd // 2, MSS)` to reflect a reduced threshold for future growth.
  - After this adjustment, the sender re-enters the **slow start phase**.
- **Fast Retransmit**
  - Because the receiver only sends cumulative ACKs, duplicate acknowledgments are used to infer packet loss.

- When the sender receives **three duplicate ACKs** for the window base sequence number, it triggers a **fast retransmit** of the missing packet.
- In this case:

```
ssthresh = max(cwnd // 2, MSS)
cwnd = ssthresh
```

- This adjustment reduces the congestion window to half its previous size (or at least 1 MSS), allowing recovery without a full timeout reset.

- **Fairness Considerations**
  - Although explicit fairness testing is not part of this project, the congestion control design inherently promotes fairness.
  - By slowing down transmission when congestion is detected (via timeouts or duplicate ACKs), the protocol avoids monopolizing bandwidth and aligns with TCP's fairness principles.

> **Note:**
>
> This design provides responsiveness to congestion events through both timeout and fast retransmit mechanisms. The configurable `ssthresh` allows flexibility in testing, while the reset-on-timeout ensures conservative recovery. Future improvements could include fast recovery or adaptive congestion window tuning to further optimize performance under diverse conditions.

> **Note:**
>
> `//` is the **floor division operator**, which divides numbers and rounds the result down to the nearest integer. This ensures that `cwnd` increments are always integer values.

Testing Methodology

To validate our Reliable Transfer Protocol (RTP), we employed a proxy module to simulate impairments such as loss, corruption, delay, and duplication, ensuring evaluation under both ideal and adverse conditions. A scenario-driven test plan combined logs and Wireshark captures to provide complementary perspectives.

Wireshark revealed control flags (SYN, ACK, FIN) from raw headers but could not interpret custom sequence or acknowledgment numbers, while logs recorded exact values and events such as corruption, drops, and retransmissions.

It should be noted that log output may not always reflect the precise chronological order of events due to Python's built-in print buffering and the concurrent use of threads. Despite this limitation, the recorded entries remain accurate in content.

Cross-referencing both sources bridged raw traces with protocol semantics, enabling reliable validation of transmission behavior and increasing confidence in our results. Log files further provide detailed runtime information about protocol events, including handshakes, data transmission, retransmissions, and error handling—with each entry following a standardized format.

> **Note:**
>
> For a comprehensive reference of all possible log events and their expected values across the sender, receiver, and connection modules, please refer to the **Log Events** section in **README.md**.

Test Environment

- **Components:** Sender, Receiver, and Proxy modules running on the same host. The Proxy introduces controlled impairments (loss, corruption, delay, duplication).
- **Tools:**
  - Wireshark for packet capture.
- **Configuration Parameters:** Please see `reliable_udp/protocol/config.py` for the complete set of options used during testing.

Protocol Invariants to be Proven per Scenario

1. Reliability

- Confirm **total comsumed bytes == total sent bytes**

2. In-Order Delivery

- `CONSUMED:` seq values increase monotonically and match sender sequence progression.
- No `CONSUMED` duplicates for the same seq (duplicate suppression).

3. Connection Lifecycle (Handshake & Teardown)

- **Handshake:**
  - Presence and order of events:
    - `SEND: syn → RECV: syn ack → SEND: ack`.
  - No data `SEND:` entries before handshake completion.
- **Teardown:**
  - `SEND: fin / RECV: fin` and final ACK sequence present.
  - `TEARDOWN: complete` logged at end.

4. Congestion Control

- `cwnd` evolution in `SEND:` log entries shows:
  - Increasing during ACKs (**slow start**).
  - Transitioning behavior after `ssthresh` (**congestion avoidance**).
- On **TIMEOUT:**
  - Logs show `cwnd` reset to MSS and `ssthresh` updated.
- On **3 duplicate ACKs:**
  - `FAST_RETRANSMIT:` entry and `cwnd` reduced to ~`ssthresh`.

5. Flow Control

- Receiver advertises `rwnd`; check advertised `rwnd` values over time.
- When `rwnd=0`:
  - Sender should stop sending application data (observe lack of non-zero seq `SEND:` entries).
  - Sender logs `PROBE:` sent entries.

- While `rwnd=0`:
  - Incoming non-zero seq packets cause receiver `DROP: seq ... rwnd 0` logs.
- Resume: after ACK with `rwnd > 0`, sender resumes normal `SEND:` entries.

6. Error Handling

- `CORRUPT:` dropped entries when checksum fails (receiver log).
- For each `CORRUPT:` or `DROP:`, a subsequent `RETRANSMIT` or `TIMEOUT` from sender should appear and later an `ACK/CONSUMED` confirms recovery.
- No `CONSUMED` for corrupted payloads; only after retransmission and successful ACK.

7. Duplicate Handling & Fast Retransmit

- Receiver logs show ack'ed packet receipts as `DROP:`; no extra `CONSUMED`.
- Sender logs show duplicate-ACK counts in `RECV: ack ... dupCount N` entries; verify duplicate counter increments.
- **Fast retransmit:**
  - When `dupCount` reaches 3, look for `FAST_RETRANSMIT:` and retransmission of `sendBase` seq.
  - After fast retransmit, observe `cwnd/ssthresh` adjustment in the same log line and subsequent ACK progression.

Test Scenarios

We executed seven controlled scenarios to validate these invariants:

Scenario	Impairment	Expected Outcome
Perfect	None	Smooth cwnd growth, complete in-order delivery
Loss	1% drop	Retransmissions on timeout/fast retransmit, cwnd halving, cumulative ACK progression
Error	0.1% corruption	Checksum detects errors, receiver drops corrupt packets, retransmissions recover data
Delay	0.01 to 0.1s latency	RTT inflation, slower cwnd growth, reduced throughput without spurious retransmits
Duplication	2% duplication	Receiver discards duplicates, fast retransmit triggered only after 3 duplicate ACKs
Low Consume	50% consumption rate	rwnd may reach 0; buffer fills and packets dropped while full; ACK progression stalls; sender probes/retransmits until application drains buffer; on rwnd>0 sender resumes
No Proxy	None	Same as Perfect: smooth cwnd growth, complete in-order delivery

> Note: For low consume, we set the receiver window (`rwnd`) to 10,000 bytes to facilitate buffer saturation under a slower application consumption rate.

Scenario Acceptance Criteria

- All defined invariants are satisfied

- Sender/receiver logs align consistently with the corresponding Wireshark capture

Evidence Collection

All scenarios were executed using the `run` bash script included in this project submission (please refer to `README.md` for usage instructions). Test artifacts are included in the project submission under `results/<scenario>`:

- **Logs**
  - Sender: `sender_out.txt`
  - Receiver: `receiver_out.txt`
- **Wireshark Captures**
  - Packet trace: `<scenario>.pcapng`

Wireshark conversation metrics were extracted from each capture via `Statistics → Conversations → UDP tab` and are presented in the Results section of this report.

Results

Invariant Verification Across Scenarios

Invariant \ Scenarios	Perfect	Loss	Error	Delay	Duplication	Low Consume	No Proxy
Reliability	✓	✓	✓	✓	✓	✓	✓
In-Order Delivery	✓	✓	✓	✓	✓	✓	✓
Connection Lifecycle	✓	✓	✓	✓	✓	✓	✓
Congestion Control	✓	✓	✓	✓	✓	✓	✓
Flow Control	✓	✓	✓	✓	✓	✓	✓
Error Handling	N/A	N/A	✓	N/A	N/A	N/A	N/A
Duplicate Handling	N/A	N/A	N/A	N/A	✓	N/A	N/A

> **Note:**
>
> N/A indicates the invariant is not applicable in that scenario.

Scenario 1: Perfect Network (No Impairment) Metrics

Flow Direction	Mapping	Packets A → B	Bytes A→B	Packets B→A	Bytes B→A	Duration (s)	Avg Throughput A→B (bits/s)	Avg Throughput B→A (bits/s)
Proxy (8001) ↔ Receiver (9002)	Proxy ↔ Receiver	62	61224	61	3172	5.42073	90355	4681

Flow Direction	Mapping	Packets A → B	Bytes A→B	Packets B→A	Bytes B→A	Duration (s)	Avg Throughput A→B (bits/s)	Avg Throughput B→A (bits/s)
Sender (9003) ↔ Proxy (8000)	Sender ↔ Proxy	62	61224	61	3172	5.4166	90424	4684

Scenario 2: Loss (1% Drop) Metrics

Flow Direction	Mapping	Packets A→B	Bytes A→B	Packets B→A	Bytes B→A	Duration (s)	Avg Throughput A→B (bits/s)	Avg Throughput B→A (bits/s)
Proxy (8001) ↔ Receiver (9002)	Proxy ↔ Receiver	67	66484	66	3432	37.539552	14168	731
Sender (9003) ↔ Proxy (8000)	Sender ↔ Proxy	73	72796	58	3016	37.535364	15515	642

Scenario 3: Error (0.1% Corruption) Metrics

Flow Direction	Mapping	Packets A→B	Bytes A→B	Packets B→A	Bytes B→A	Duration (s)	Avg Throughput A→B (bits/s)	Avg Throughput B→A (bits/s)
Proxy (8001) ↔ Receiver (9002)	Proxy ↔ Receiver	63	62276	61	3172	15.403948	32342	1647
Sender (9003) ↔ Proxy (8000)	Sender ↔ Proxy	63	62276	61	3172	15.404006	32342	1647

Scenario 4: Delay (0.01–0.1s Latency) Metrics

Flow Direction	Mapping	Packets A→B	Bytes A→B	Packets B→A	Bytes B→A	Duration (s)	Avg Throughput A→B (bits/s)	Avg Throughput B→A (bits/s)
Proxy (8001) ↔ Receiver (9002)	Proxy ↔ Receiver	138	141176	137	7124	224.99	5019	253
Sender (9003) ↔ Proxy (8000)	Sender ↔ Proxy	138	141176	137	7124	225.46	5009	252

Scenario 5: Duplication (2% Duplication) Metrics

Flow Direction	Mapping	Packets A→B	Bytes A→B	Packets B→A	Bytes B→A	Duration (s)	Avg Throughput A→B (bits/s)	Avg Throughput B→A (bits/s)
Proxy (8001) ↔ Receiver (9002)	Proxy ↔ Receiver	90	89680	89	4628	7.406298	96868	4998
Sender (9003) ↔ Proxy (8000)	Sender ↔ Proxy	68	67536	110	5720	7.400785	73004	6183

Scenario 6: Low Consume (50% Application Consumption Rate) Metrics

Flow Direction	Mapping	Packets A→B	Bytes A→B	Packets B→A	Bytes B→A	Duration (s)	Avg Throughput A→B (bits/s)	Avg Throughput B→A (bits/s)
Sender (9003) ↔ Proxy (8000)	Sender ↔ Proxy	182	96138	181	9412	124.239	6190	606
Proxy (8001) ↔ Receiver (9002)	Proxy ↔ Receiver	182	96138	181	9412	124.240	6190	606

Scenario 7: No Proxy Metrics

Flow Direction	Role Mapping	Packets A→B	Bytes A→B	Packets B→A	Bytes B→A	Duration (s)	Avg Throughput A→B (bits/s)	Avg Throughput B→A (bits/s)
9003 ↔ 9002	Sender ↔ Receiver	61	60172	60	3120	11.363	42361	2196

Discussion

Our Reliable Transfer Protocol (RTP) successfully implements a connection-oriented, reliable, pipelined transfer over UDP with flow and congestion control, as validated by Wireshark captures and logs across seven scenarios. Key features including three-way handshake/teardown, CRC32 checksums, cumulative ACKs, timeout/fast retransmit, sliding window (cwnd/rwnd), slow start/congestion avoidance, ensure reliability (matched sent/consumed bytes), in-order delivery (monotonic sequences, no duplicates), and adaptability.

Correctness of Protocol Features

Our protocol establishes connection-oriented communication through a three-way handshake (SYN, SYN-ACK, ACK) and a four-way teardown (FIN, ACK, FIN, ACK), as evidenced in the logs and Wireshark captures across all scenarios. For instance, in the Perfect Network scenario, the handshake completes without issues.

Sender Log (default/sender_out.txt):

```
SOCKET: listening from  0.0.0.0  at local port  9003
SEND: syn
RECV: syn ack
SENT: ack
INIT: Connected to ('127.0.0.1', 8000)
```

Receiver Log (default/receiver_out.txt):

```
SOCKET: listening from  0.0.0.0  at local port  9002
RECV: syn
SEND: syn ack
RECV: ack
INIT: connected to ('127.0.0.1', 8001)
```

The teardown logs consistently show "TEARDOWN: complete," indicating graceful closure.

Sender Log (default/sender_out.txt):

```
SEND: fin
RECV: ack
RECV: fin
```

```
SEND: final ack
TEARDOWN: complete
```

Receiver Log (default/receiver_out.txt):

```
RECV: fin
SEND: ack
SEND: fin
RECV: final ack
TEARDOWN: complete
```

However, as noted in the specification, the handshake assumes a stable network and lacks retry mechanisms, which could lead to failures in high-loss environments. This simplicity ensures correctness in controlled tests but may not be fully robust in real-world settings.

Reliability is proven by the consistent matching of total sent and consumed bytes in all scenarios, with in-order delivery verified through monotonically increasing sequence numbers in "CONSUMED:" logs and no duplicates.

Receiver Log (default/receiver_out.txt):

```
SEND: ack 1000, rwnd 64000
RECV: seq 1000
SEND: ack 1000, rwnd 63000
RECV: seq 2000
SEND: ack 1000, rwnd 62000
CONSUMED: seq 1000
CONSUMED: seq 2000
...
SEND: ack 3000, rwnd 64000
RECV: seq 3000
CONSUMED: seq 3000
...
RECV: seq 4000
SEND: ack 4000, rwnd 63000
CONSUMED: seq 4000
RECV: seq 5000
SEND: ack 5000, rwnd 63000
RECV: seq 6000
CONSUMED: seq 5000
CONSUMED: seq 6000
SEND: ack 7000, rwnd 64000
RECV: seq 7000
CONSUMED: seq 7000
SEND: ack 7000, rwnd 63000
RECV: seq 8000
CONSUMED: seq 8000
```

Simplified to just CONSUMED logs:

```
CONSUMED: seq 1000
CONSUMED: seq 2000
...
CONSUMED: seq 3000
...
CONSUMED: seq 4000
...
CONSUMED: seq 5000
CONSUMED: seq 6000
...
CONSUMED: seq 7000
...
CONSUMED: seq 8000
```

As stated, we can see that sequence packets are consumed in order.

The use of CRC32 checksums effectively detects corruption, as seen in the Error scenario, where corrupted packets are dropped ("CORRUPT:" logs) and recovered via retransmissions without data loss.

Sender Log (error/sender_out.txt):

```
SEND: seq 19000 cwnd 2629 ssthresh 2168 dupCount 0
SEND: seq 20000 cwnd 2629 ssthresh 2168 dupCount 0
SEND: seq 21000 cwnd 2629 ssthresh 2168 dupCount 0
RECV: ack 19000 dupCount 0
RECV: ack 19000 dupCount 1
TIMEOUT: seq 19000 cwnd 2629 ssthresh 2168 dupCount 2
RETRANSMIT: seq 19000 cwnd 1000 ssthresh 1314 dupCount 0
RECV: ack 21000 dupCount 0
SEND: seq 22000 cwnd 2000 ssthresh 1314 dupCount 0
CORRUPT: dropped                                          <-- Corruption detected
TIMEOUT: seq 21000 cwnd 2000 ssthresh 1314 dupCount 0
RETRANSMIT: seq 21000 cwnd 1000 ssthresh 1000 dupCount 0
RECV: ack 21000 dupCount 0
TIMEOUT: seq 21000 cwnd 1000 ssthresh 1000 dupCount 1
RETRANSMIT: seq 21000 cwnd 1000 ssthresh 1000 dupCount 0
RECV: ack 23000 dupCount 0
```

Receiver Log (error/receiver_out.txt):

```
CONSUMED: seq 15000
CONSUMED: seq 16000
CONSUMED: seq 17000
CONSUMED: seq 18000
OUT_OF_RWND_RANGE
DROP: seq 15000, rwnd 64000
SEND: ack 19000, rwnd 64000
RECV: seq 19000
SEND: ack 19000, rwnd 63000
```

```
RECV: seq 20000
SEND: ack 19000, rwnd 62000
CORRUPT: dropped               <-- Corruption detected
CONSUMED: seq 19000
CONSUMED: seq 20000
OUT_OF_RWND_RANGE
DROP: seq 19000, rwnd 64000
SEND: ack 21000, rwnd 64000
RECV: seq 22000
SEND: ack 21000, rwnd 63000
RECV: seq 21000
CONSUMED: seq 21000
CONSUMED: seq 22000
```

We can see in the receiver logs that recovery through retransmission was successful, as we observe a corruption detection event, followed immediately by the CONSUMED sequence numbers increasing continuously without gaps. If the retransmission had failed, the consumer process would have halted or skipped a sequence number.

Cumulative ACKs and timeout-based retransmissions of the window base packet minimize overhead, but this approach can be inefficient in high-loss conditions, e.g. in the Loss scenario, retransmissions led to cwnd resets and reduced throughput (from ~90 kbps in Perfect to ~15 kbps).

Fast retransmit, triggered by >= three duplicate ACKs (See the Limitations and Future Improvements section for a detailed discussion of this implementation quirk), was observed in Duplication and Loss scenarios, correctly reducing cwnd to half and promoting quicker recovery than timeouts alone.

Sender Log (duplicate/sender_out.txt):

```
RECV: ack 6000 dupCount 0
RECV: ack 6000 dupCount 1
RECV: ack 6000 dupCount 2
FAST_RETRANSMIT: seq 6000 cwnd 2500 ssthresh 1000 dupCount 3
```

Sender Log (loss/sender_out.txt):

```
RECV: ack 19000 dupCount 0
...
RECV: ack 19000 dupCount 1
RECV: ack 19000 dupCount 2
FAST_RETRANSMIT: seq 19000 cwnd 5193 ssthresh 1000 dupCount 3
```

Pipelining is effectively implemented via the sliding window, with cwnd starting at one MSS and growing during slow start and congestion avoidance phases. Logs show an observable exponential growth in slow start transitioning to linear increments once ssthresh is reached, as in the Perfect scenarios.

Sender Log (default/sender_out.txt):

```
RETRANSMIT: seq 1000 cwnd 1000 ssthresh 1000 dupCount 0
RECV: ack 3000 dupCount 0
SEND: seq 3000 cwnd 2000 ssthresh 1000 dupCount 0
SEND: seq 4000 cwnd 2000 ssthresh 1000 dupCount 0
RECV: ack 3000 dupCount 0
RECV: ack 4000 dupCount 1
SEND: seq 5000 cwnd 2500 ssthresh 1000 dupCount 0
SEND: seq 6000 cwnd 2500 ssthresh 1000 dupCount 0
RECV: ack 5000 dupCount 0
RECV: ack 6000 dupCount 0
SEND: seq 7000 cwnd 3244 ssthresh 1000 dupCount 0
SEND: seq 8000 cwnd 3244 ssthresh 1000 dupCount 0
SEND: seq 9000 cwnd 3244 ssthresh 1000 dupCount 0
```

This allows multiple packets in flight, contributing to high throughput (~90 kbps in Perfect). However, in impaired scenarios like Delay, the inflated RTT (up to 225 seconds total duration) slows cwnd growth, resulting in lower throughput (~5 kbps), which is expected.

Flow control prevents receiver buffer overflow by advertising rwnd in ACKs, with the sender halting data transmission when rwnd=0 and using probes to resume. The Low Consume scenario critically tests this with a 50% consumption rate and rwnd=10,000 bytes.

Receiver Log (low-consume/receiver_out.txt):

```
SEND: ack 11000, rwnd 2000
RECV: seq 11000
SEND: ack 11000, rwnd 1000
RECV: seq 12000
SEND: ack 11000, rwnd 0       <-- Receiver halting sender
RECV_BUF_FULL
DROP: seq 11000 rwnd 0
OUT_OF_RWND_RANGE
DROP: seq 0, rwnd 0
SEND: ack 11000, rwnd 0
OUT_OF_RWND_RANGE
DROP: seq 0, rwnd 0
SEND: ack 11000, rwnd 0
CONSUMED: seq 11000
CONSUMED: seq 12000
OUT_OF_RWND_RANGE
DROP: seq 0, rwnd 2000
SEND: ack 13000, rwnd 2000   <-- Receiver resumes receiving
```

Sender Log (low-consume/sender_out.txt):

```
RECV: ack 11000 dupCount 8
PROBE: sent
SEND: seq 11000 cwnd 2000 ssthresh 1000 dupCount 0
RECV: ack 11000 dupCount 0
```

```
RECV: ack 11000 dupCount 1
SEND: seq 12000 cwnd 2000 ssthresh 1000 dupCount 2
RECV: ack 11000 dupCount 2
FAST_RETRANSMIT: seq 11000 cwnd 2000 ssthresh 1000 dupCount 3
RECV: ack 11000 dupCount 3
RECV_BUF_FULL: stop                                          <-- Sender halts
sending
PROBE: sent
RECV_BUF_FULL: stop
RECV: ack 11000 dupCount 4
PROBE: sent
RECV_BUF_FULL: stop
RECV: ack 11000 dupCount 5
PROBE: sent
RECV_BUF_FULL: stop
RECV: ack 13000 dupCount 6
RECV: ack 13000 dupCount 0
PROBE: sent
SEND: seq 13000 cwnd 2000 ssthresh 1000 dupCount 1             <-- Sender resumes
sending
RECV: ack 13000 dupCount 1
```

(Note: For a detailed discussion of the observed dupCount of 8, please see the Limitations and Future Improvements section.)

Throughput drops significantly (~6 kbps) in the Low Consume scenario, but delivery remains reliable and in-order once resumed.

Receiver Log (low-consume/receiver_out.txt):

(Simplified to just CONSUMED entires)

```
CONSUMED: seq 13000
CONSUMED: seq 14000
...
CONSUMED: seq 15000
CONSUMED: seq 16000
...
CONSUMED: seq 17000
CONSUMED: seq 18000
```

Congestion control adapts to network conditions via slow start, avoidance, timeouts, and fast retransmit, promoting efficient bandwidth use. In Loss scenarios, cwnd halving and ssthresh updates prevent congestion collapse, with throughput reductions reflecting appropriate backoff (e.g., ~15 kbps in Loss).

Sender Log (loss/sender_out.txt):

```
SEND: seq 24000 cwnd 5193 ssthresh 1000 dupCount 1
RECV: ack 19000 dupCount 1
RECV: ack 19000 dupCount 2
```

```
FAST_RETRANSMIT: seq 19000 cwnd 5193 ssthresh 1000 dupCount 3
TIMEOUT: seq 19000 cwnd 2596 ssthresh 2596 dupCount 3
```

While we did not conduct explicit fairness testing (e.g., against competing TCP flows), the design's alignment with TCP AIMD principles suggests inherent fairness. Future tests with network simulators could quantify this.

Performance Analysis and Acceptability

Wireshark metrics indicate acceptable performance in ideal conditions, with throughputs of ~90 kbps in Perfect and ~42 kbps in No Proxy, considering our local host setup and MSS of 1,000 bytes.

Impairments naturally degrade performance: Loss and Error reduce throughput due to retransmissions, Delay due to RTT inflation, and Duplication due to extra ACK overhead.

In a controlled environment with simulated losses via proxy, the protocol maintains reliability without excessive overhead, meeting the project's requirement for acceptable performance.

Limitations and Future Improvements

While the protocol is correct and functional, reliance on cumulative acknowledgments can lead to unnecessary retransmissions of packets that have already been received in order.

The current design retransmits only the base packet. This simplifies implementation but may perform poorly under consecutive packet loss, where selective retransmission would be more efficient.

Both the handshake and probe mechanisms are static and lack adaptivity. They do not adjust to varying network conditions, which limits resilience in dynamic environments.

The current implementation supports only one-to-one communication. Extending the protocol to handle one-to-many or many-to-many scenarios would broaden its applicability.

There is an off-by-one error (in rdt_sender.py) that causes Fast Retransmit to trigger on the 4th duplicate ACK instead of the standard 3rd. From the results we observed, this slight delay does not impede the main purpose of fast retransmission, which is to reduce reliance on timeouts and achieve quicker recovery. Therefore, we have listed this as a Future Improvement.

Furthermore, due to the asynchronous nature of the multithreaded design, the main logic thread occasionally registered a burst of duplicate ACKs (e.g., up to 8) before executing the retransmission check. This is a concurrency artifact and not a bug in the threshold logic itself, as the primary goal of the Fast Retransmit mechanism is to track duplicate ACKs and initiate quicker recovery. Despite these minor threshold differences, the underlying concept is sound.

Conclusion

Our RTP meets the mini project's requirements for a pipelined, reliable protocol with flow and congestion control, as validated through rigorous testing and analysis. The results affirm its correctness in diverse scenarios, providing a foundation for reliable communication over UDP while identifying opportunities for refinement.

References

- Computer Networking: A Top-Down Approach (Kurose & Ross)
- RFC 793 - Transmission Control Protocol (TCP)
- RFC 5681 - TCP Congestion Control